

EIF200 – Fundamentos de Informática

Tutoría #4 - Ciclo II 2026

Miércoles 19 de agosto de 2026

Tutor:

Cipriano Rivera Escobar

Índice

1. Funciones y subrutinas	2
1.1. Entrada, proceso y salida	2
2. Organización del código: archivos .h y .cpp	2
3. Código de caracteres ASCII	4
4. Ejercicios	4
4.1. Ejercicio 1 – Contar caracteres	4
4.2. Ejercicio 3 – Convertir a mayúsculas	5
4.3. Ejercicio 4 – Convertir a minúsculas	6
4.4. Ejercicio 5 – Determinar si una palabra es palíndroma	6
4.5. Ejercicio 6 – Capitalizar una palabra	7
4.6. Ejercicio 7 – Invertir un número	7
4.7. Ejercicio 8– Convertir texto a int	8
4.8. Ejercicio 9 – Determinar el tipo de triángulo	8
4.9. Ejercicio 10 – Piedra, papel o tijera	9

1. Funciones y subrutinas

Una función es un bloque de instrucciones que permite realizar una tarea específica dentro de un programa. Una de sus principales ventajas es que nos permite organizar mejor el código y reutilizar instrucciones cuando sea necesario.

1.1. Entrada, proceso y salida

Al analizar un problema, podemos dividirlo en tres partes principales: qué información necesitamos, qué debemos hacer con esa información y qué resultado esperamos obtener.

Subrutinas

- ❑ Entradas = Parámetros (por valor).
- ❑ Proceso = Cuerpo de la subrutina (algoritmo/lógica).
- ❑ Salida = Valor que se retorna.

```
int calculateArea(int side) {  
    int area = side * side;  
    return area;  
}
```



Figura 1: Entrada, proceso y salida de una subrutina.

Esta misma idea puede aplicarse al trabajar con funciones:

- Los datos de **entrada** pueden recibirse mediante parámetros.
- El **proceso** se realiza dentro de la función.
- La **salida** puede devolverse mediante **return**.

2. Organización del código: archivos .h y .cpp

Cuando un programa comienza a crecer, puede resultar difícil mantener todo el código en un solo archivo. Por esta razón, podemos separar nuestro programa en diferentes archivos.

En C++, una forma común de organizar las funciones es utilizar un archivo **.h** y un archivo **.cpp**. Aunque ambos están relacionados, cada uno tiene un propósito diferente:

- El archivo **.h** indica **qué funciones existen** y qué información necesitan para funcionar.

- El archivo `.cpp` contiene **cómo funcionan esas funciones**, es decir, las instrucciones que realizan el trabajo.

Podemos verlo de una manera sencilla:

`.h` → ¿Qué funciones tenemos?

`.cpp` → ¿Cómo funcionan esas funciones?

La siguiente imagen muestra cómo podemos separar una función entre ambos archivos:

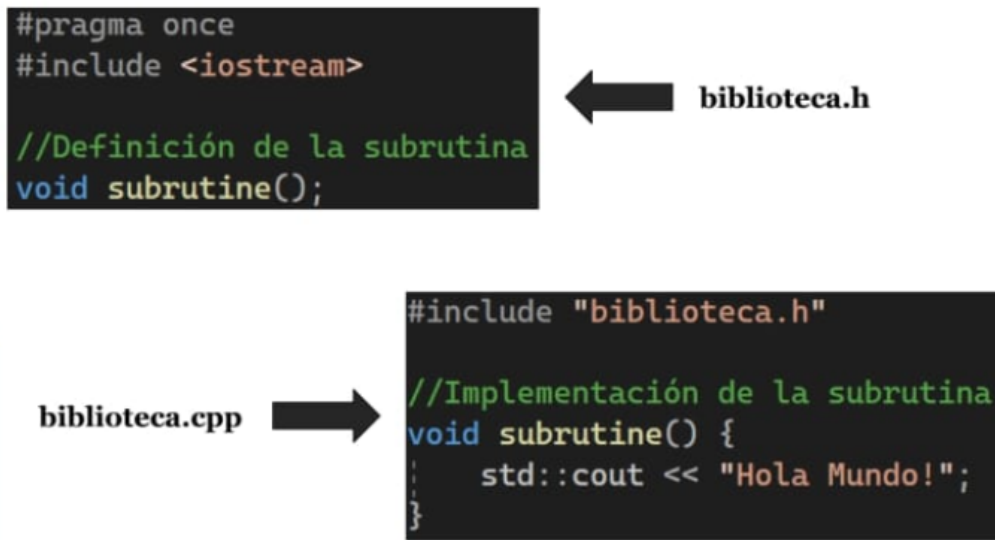


Figura 2: Separación de una función en archivos `.h` y `.cpp`.

De esta manera, podemos mantener el código más ordenado y facilitar su lectura y mantenimiento.

3. Código de caracteres ASCII

Caracteres ASCII de control			Caracteres ASCII imprimibles			ASCII extendido (Página de código 437)										
00	NULL	(carácter nulo)	32	espacio	64	@	96	`	128	Ç	160	à	192	Ł	224	Ó
01	SOH	(inicio encabezado)	33	!	65	A	97	a	129	ü	161	í	193	ł	225	ô
02	STX	(inicio texto)	34	"	66	B	98	b	130	ë	162	ó	194	Ł	226	õ
03	ETX	(fin de texto)	35	#	67	C	99	c	131	â	163	ú	195	ł	227	ö
04	EOT	(fin transmisión)	36	\$	68	D	100	d	132	ä	164	ñ	196	Ł	228	ø
05	ENQ	(consulta)	37	%	69	E	101	e	133	å	165	Ñ	197	ł	229	õ
06	ACK	(reconocimiento)	38	&	70	F	102	f	134	ä	166	ª	198	Ł	230	µ
07	BEL	(timbre)	39	'	71	G	103	g	135	ç	167	º	199	Ł	231	þ
08	BS	(retroceso)	40	(	72	H	104	h	136	ê	168	¿	200	ł	232	ÿ
09	HT	(tab horizontal)	41	)	73	I	105	i	137	è	169	©	201	Ł	233	ù
10	LF	(nueva línea)	42	*	74	J	106	j	138	ê	170	¬	202	ł	234	ú
11	VT	(tab vertical)	43	+	75	K	107	k	139	ï	171	½	203	Ł	235	û
12	FF	(nueva página)	44	,	76	L	108	l	140	î	172	¾	204	ł	236	ý
13	CR	(retorno de carro)	45	-	77	M	109	m	141	ï	173	ı	205	Ł	237	ÿ
14	SO	(desplaza afuera)	46	.	78	N	110	n	142	À	174	«	206	Ł	238	—
15	SI	(desplaza adentro)	47	/	79	O	111	o	143	Á	175	»	207	ł	239	·
16	DLE	(esc.vínculo datos)	48	0	80	P	112	p	144	Ê	176	°	208	Ł	240	=
17	DC1	(control disp. 1)	49	1	81	Q	113	q	145	æ	177	±	209	ł	241	±
18	DC2	(control disp. 2)	50	2	82	R	114	r	146	Æ	178	²	210	Ł	242	²
19	DC3	(control disp. 3)	51	3	83	S	115	s	147	ø	179	³	211	ł	243	³
20	DC4	(control disp. 4)	52	4	84	T	116	t	148	ö	180	´	212	Ł	244	´
21	NAK	(conf. negativa)	53	5	85	U	117	u	149	ò	181	Å	213	ł	245	§
22	SYN	(inactividad sinc)	54	6	86	V	118	v	150	û	182	Ä	214	Ł	246	÷
23	ETB	(fin bloque trans)	55	7	87	W	119	w	151	ü	183	Å	215	ł	247	¸
24	CAN	(cancelar)	56	8	88	X	120	x	152	ý	184	©	216	Ł	248	°
25	EM	(fin del medio)	57	9	89	Y	121	y	153	Ö	185	ª	217	ł	249	ˆ
26	SUB	(sustitución)	58	:	90	Z	122	z	154	Ü	186	ı	218	Ł	250	ˆ
27	ESC	(escape)	59	;	91	[	123	{	155	ø	187	ı	219	ł	251	ˆ
28	FS	(sep. archivos)	60	<	92	\	124		156	£	188	ı	220	Ł	252	ˆ
29	GS	(sep. grupos)	61	=	93	]	125	}	157	Ø	189	¢	221	ł	253	ˆ
30	RS	(sep. registros)	62	>	94	^	126	~	158	×	190	¥	222	Ł	254	ˆ
31	US	(sep. unidades)	63	?	95	_			159	f	191	ı	223	ł	255	nbsp
127	DEL	(suprimir)														

Figura 3: Código de caracteres ASCII.

4. Ejercicios

4.1. Ejercicio 1 – Contar caracteres

Crear una función que reciba una palabra y determine cuántos caracteres contiene.

Ejemplo:

Si el usuario ingresa:

hola

La función debe indicar:

La palabra tiene 4 caracteres.

Ejemplo:

Si el usuario ingresa:

costa rica

La función debe indicar:

acir atsoc

4.2. Ejercicio 3 – Convertir a mayúsculas

- Realizar una rutina en C++ que reciba como parámetro una cadena de caracteres y devuelva la cadena de caracteres en Mayúsculas. La rutina debe funcionar para cualquier tipo y tamaño de texto.

- Por ejemplo:

“Hola, Mundo” u “hola, mundo”

a

“HOLA, MUNDO”

Figura 4: Ejercicio 3: convertir una palabra a mayúsculas.

4.3. Ejercicio 4 – Convertir a minúsculas

- Realizar una rutina en C++ que reciba como parámetro una cadena de caracteres y devuelva la cadena de caracteres en Minúsculas. La rutina debe funcionar para cualquier tipo y tamaño de texto.

- Por ejemplo:

“Hola, Mundo” u “HOLA, MUNDO”

a

“hola, mundo”

Figura 5: Ejercicio 4: convertir una palabra a minúsculas.

4.4. Ejercicio 5 – Determinar si una palabra es palíndroma

Crear una función que reciba una palabra y determine si es un palíndromo.

Una palabra es palíndroma cuando se lee de la misma manera de izquierda a derecha y de derecha a izquierda.

Ejemplo:

Si el usuario ingresa:

reconocer

La función debe indicar:

La palabra es palíndroma.

Si el usuario ingresa:

casa

La función debe indicar:

La palabra no es palíndroma.

4.5. Ejercicio 6 – Capitalizar una palabra

Realizar una rutina en C++ que reciba como parámetro una cadena de caracteres y devuelva capitalizado el texto. La rutina debe funcionar para cualquier tipo y tamaño de texto.

Por ejemplo:

“La imaginacion es mas importante que el
conocimiento”

a

“La Imagination Es Mas Importante Que El
Conocimiento”

Figura 6: Ejercicio 6: capitalizar una palabra.

4.6. Ejercicio 7 – Invertir un número

Crear una función que reciba un número entero y lo devuelva con sus dígitos en orden inverso.

Ejemplo:

Si el usuario ingresa:

12345

La función debe indicar:

54321

4.7. Ejercicio 8– Convertir texto a int

- Realizar una rutina en C++ que reciba como parámetro una cadena de caracteres que representa un número en decimal y retorne el número en entero. La rutina debe funcionar para cualquier texto que represente un número.
- Por ejemplo:

“6214” a 6214

Figura 7: Ejercicio 8: Convertir texto a int

4.8. Ejercicio 9 – Determinar el tipo de triángulo

Crear una función que reciba los tres lados de un triángulo y determine qué tipo de triángulo es.

Antes de determinar el tipo de triángulo, se debe comprobar que los tres lados realmente puedan formar un triángulo.

Para que tres lados puedan formar un triángulo deben cumplir la **desigualdad triangular**. Esto significa que la suma de dos lados siempre debe ser mayor que el tercer lado.

Las condiciones que se deben comprobar son:

- $\text{lado 1} + \text{lado 2} > \text{lado 3}$
- $\text{lado 1} + \text{lado 3} > \text{lado 2}$
- $\text{lado 2} + \text{lado 3} > \text{lado 1}$

Si las tres condiciones se cumplen, los lados pueden formar un triángulo.

Ejemplo:

Si el usuario ingresa:

5, 5, 8

Se debe comprobar:

$5 + 5 > 8 \rightarrow \text{Verdadero}$

$5 + 8 > 5 \rightarrow \text{Verdadero}$

$5 + 8 > 5 \rightarrow \text{Verdadero}$

Por lo tanto, los lados sí pueden formar un triángulo.

Una vez comprobado esto, se debe determinar qué tipo de triángulo es:

- **Equilátero:** los tres lados son iguales.
- **Isósceles:** dos lados son iguales.
- **Escaleno:** los tres lados son diferentes.

Ejemplos:

5, 5, 5 → Triángulo equilátero

5, 5, 8 → Triángulo isósceles

4, 5, 6 → Triángulo escaleno

Si las condiciones para formar un triángulo no se cumplen, se debe indicar:

Los lados no forman un triángulo.

4.9. Ejercicio 10 – Piedra, papel o tijera

Crear una función que reciba la elección de dos jugadores y determine quién gana la partida.

Las opciones serán:

1 → Piedra

2 → Papel

3 → Tijera

Las reglas del juego son:

- La **piedra** gana a la tijera.
- La **tijera** gana al papel.
- El **papel** gana a la piedra.
- Si ambos jugadores eligen la misma opción, hay un empate.

Ejemplo:

Si el jugador 1 selecciona:

1 – Piedra

Y el jugador 2 selecciona:

3 - Tijera

El programa debe indicar:

Gana el jugador 1.

Otro ejemplo:

Si ambos jugadores seleccionan:

2 - Papel

El programa debe indicar:

Empate.

Además, el programa debe comprobar que las opciones ingresadas sean válidas. Si un jugador introduce un número diferente de 1, 2 o 3, se debe indicar que la opción no es válida.